

MANUAL DE TESTES – BlindRevoke

Para reproduzir o teste automatizado que está referenciado no paper na seção 6 “Avaliação experimental” você primeiro precisa preparar o ambiente: será necessário instalar a rede VON network e depois subir o serviço de *bloom filters*.

O Github do projeto é: <https://github.com/arlindoconceicao/vertex>

A instalação que apresentamos é para linux Mint mas pode ser adaptada facilmente para outras distribuições.

Acompanhe todos estes passos seguindo o roteiro a seguir:

1. Instalar a rede VON Network

Para usar o software é necessário primeiro instalar e inicializar a rede VON network (que será o ledger que vai gravar os DIDs, Schemas e CredDefs).

VON network é uma rede Hyperledger Indy portátil, muito utilizada para desenvolvedores que precisam de um ecossistema de Identidade Descentralizada (DID) para testes locais.

Como ela é executada a partir do Docker, o processo no Linux é relativamente direto, mas exige que o ambiente esteja bem configurado. Antes de começar, certifique-se de que seu sistema tem as ferramentas necessárias instaladas. Aqui está o passo a passo:

Git

Docker (versão 17.05 ou superior)

Para instalar o git use:

```
sudo apt update
sudo apt install git
git --version
```

Para verificar se você já tem o docker instalado digite:

```
docker --version
```

Caso não tenha o docker instalado digite os seguintes comandos:

a) Remova versões antigas:

```
sudo apt-get remove docker docker-engine docker.io containerd runc
```

b) Instale as dependências necessárias

```
sudo apt-get update
sudo apt-get install ca-certificates curl gnupg
```

c) Adicione a chave GPG oficial do Docker

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

d) Adicione o repositório do Docker (O truque do Mint)

O comando abaixo lê o arquivo do seu sistema, descobre qual é o UBUNTU_CODENAME correto (por exemplo, jammy ou focal) e adiciona o repositório certo:

```
echo \ "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \ $(. /etc/os-release && echo
"$UBUNTU_CODENAME") stable" | \ sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

e) Instale o Docker

```
sudo apt-get update sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
```

Caso aconteça algum problema pesquise sobre a instalação do docker no seu sistema operacional específico. Este procedimento de instalação do docker é no sistema operacional Linux Mint. Em nossos testes estamos usando o Docker version 29.4.0.

1.1 Clonando o Repositório

Abra o terminal e baixe o código oficial do projeto:

```
git clone https://github.com/bcgov/von-network.git cd von-network
```

1.2 Construindo a Imagem (Build)

Dentro da pasta do projeto, você precisa construir a imagem Docker que contém os nós da rede Indy:

```
./manage build
```

1.3 Iniciando a Rede

Existem duas formas principais de rodar a rede:

```
./manage start ou ./manage start --logs (com logs)
```

Em geral é mais prático iniciar a rede com `./manage start`.

1.4 Acessando a Interface Web

Uma das melhores funcionalidades da VON network é o Web Server que ela disponibiliza. Após iniciar, abra o seu navegador e acesse:

<http://localhost:9000>

Nesta interface você vai ter suporte as seguintes operações:

- Ver o status dos nós da rede.
- Visualizar o Genesis File (necessário para conectar agentes como o ACA-Py ou outros aplicativos). Para ver o arquivo Genesis acesse <http://localhost:9000/genesis>.
- Registrar DIDs diretamente no Ledger.

1.5 Comandos Úteis de Gerenciamento

Ação	Comando
Parar a rede	<code>./manage stop</code>
Limpar dados e reiniciar	<code>./manage down</code>
Ver logs	<code>./manage logs</code>

1.6 Telas da rede VON Network

Aqui estão os prints de algumas telas da rede VON network:

The screenshot displays the VON Network interface in a web browser. The address bar shows 'localhost:9000'. The page has a dark blue header with the text 'localhost'. Below the header, there are two main sections:

- Validator Node Status:** This section lists four nodes (Node1, Node2, Node3, Node4) with their respective details. Each node entry includes a circular icon, a DID, Uptime, Txns, and Indy-node version. Below the list, there is a link to 'View detailed information about the status of the running validator nodes:' and a button labeled 'Detailed Status'.
- Connect to the Network:** This section provides instructions to download the genesis transaction file. It includes a button labeled 'Genesis Transaction' and a 'JSON' label.
- Authenticate a New DID:** This section allows users to register a new DID. It includes a heading 'Authenticate a New DID', a sub-heading 'Easily write a new DID to the ledger for new identity owners.', and two radio buttons: 'Register from seed' (selected) and 'Register from DID'. Below these are input fields for 'Wallet seed (32 characters or base64)', 'DID (optional)', and 'Alias (optional)'. There is also a 'Role' dropdown menu with 'Endorser' selected and a 'Register DID' button.

localhost:9000/browse/domain

localhost

Back to Status

Ledger Transactions

Domain

28 record(s)

Type: (Select)

Filter:

123

#1TransactionType: NYM
Nym: V45GRU86Z58d6TV7PBue6f
Role: TRUSTEE
Verkey: ~CoRE63DVYmZtK8uAzNdxRaw Data

#2MetadataFrom nym: V45GRU86Z58d6TV7PBue6fTransactionType: NYM
Nym: Th7HpTaRZVRynPiabds81Y
Role: STEWARD
Verkey: ~TTYfekw4GlagBn8VCqPJ1CRaw Data

localhost:9000/browse/domain?page=1&query=&txn_type=101

localhost

Back to Status

Ledger Transactions

Domain

3 record(s)

Type: SCHEMA

Filter:

Clear filter

1

#8Message WrapperTransaction ID: 32gWkUokJGkoF1vD2ANDqv:2:cpf:1.8
Transaction time: 28/04/2026, 09:10:43 (1776687843)
Signed by: 32gWkUokJGkoF1vD2ANDqvMetadataFrom nym: 32gWkUokJGkoF1vD2ANDqv
Request ID: 1776687843612177788
Digest: 0ca1b3d642055b681b6f0178b6e7dd7d773e7f217f2318a208521a64f51e855TransactionType: SCHEMA
Schema name: cpf
Schema version: 1.8
Schema attributes:

- root_merkle_1
- seed
- time_window
- unit_of_time
- none
- idade
- cpf
- start_time

localhost:9000/browse/domain?page=1&query=&txn_type=102

localhost

Back to Status

Ledger Transactions

Domain

3 record(s)

Type: CRED_DEF

Filter:

Clear filter

1

#11Message WrapperTransaction ID: 32gWkUokJGkoF1vD2ANDqv:3:CL:18:TAG1
Transaction time: 28/04/2026, 09:12:26 (1776687146)
Signed by: 32gWkUokJGkoF1vD2ANDqvMetadataFrom nym: 32gWkUokJGkoF1vD2ANDqv
Request ID: 1776687146805464800
Digest: 48206d946a2d59a5aacc8afae405af2b6fb57cb04e5a8ee864d2b01e4ffeb59aTransactionType: CRED_DEF
Reference: 18
Signature type: CL
Tag: TAG1
Attributes:

- carteira_motorista
- master_secret
- none
- observacoes
- root_merkle_1
- seed
- start_time
- time_window
- unit_of_time

Raw Data

2. Subir o serviço dos Bloom Filters

Para que o serviço de revogação funcione é necessário executar o serviço dos *bloom filters*. Este serviço permite controlar a revogação e a consulta de credenciais revogadas.

2.1 Faça clone da pasta “bfilter”

No Github do projeto faça o clone da pasta “bfilter” que é o serviço de bloom filter. Você precisa ter instalado antes o Rust e o npm. Caso não tenha digite os comandos abaixo:

2.1.1 Instalação do RUST

```
curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Quando o instalador abrir, normalmente basta escolher a opção padrão. Depois, carregue o ambiente no shell atual com:

```
source "$HOME/.cargo/env"
```

Para confirmar que deu certo, rode:

```
rustc --version  
cargo --version  
rustup --version
```

Se quiser atualizar depois, use:

```
rustup update
```

Nós estamos usando a versão **rustc 1.95.0 (59807616e 2026-04-14)**.

2.1.2 Instalação do node + npm

Passo 1: Atualize a lista de pacotes:

```
sudo apt update
```

Passo 2: Instale o Node.js e o npm:

```
sudo apt install nodejs npm -y
```

Passo 3: Verifique as versões:

```
node -v  
npm -v
```

Estamos usando a versão **v18.19.1** para node e **9.2.0** para npm.

2.2 Subindo o serviço de bloom filter

Depois de clonar a pasta “bfilter” no repositório você deve entrar nesta pasta e digitar os seguintes

comandos:

Abra um novo terminal e digite:

```
BFILTER_ENABLE_TEST_API=1 BFILTER_ADMIN_TOKEN="dev-admin-token" cargo run
```

Você deve ver no terminal algo como:

```
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.31s
Running `target/debug/bfilter`

=====
bfilter service listening on http://127.0.0.1:8080
mode: TEST
warning: test-only endpoints are ENABLED
public endpoints: GET /health, GET /manifest, GET /filters/:filter_id, GET /filters/for-window/:window_start, POST /check
admin endpoints: POST /admin/revocations, POST /admin/revocations/v2, POST /admin/filters/close, POST /admin/filters/create, POST /admin/filters/rotate
test-only endpoints: POST /test/reset
test hint: use this mode only in controlled validation environments
=====
```

Abra o Browser no endereço <http://127.0.0.1:8080/manifest> e você verá algo semelhante a isto:

```
← → ↻ ⓘ 127.0.0.1:8080/manifest
Estilos de formatação ☒

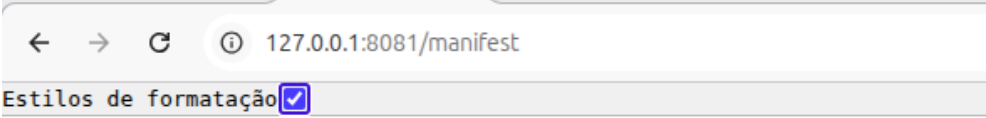
{
  "manifest": {
    "active_filter_id": "filter-1776432374706800409",
    "false_positive_power": 32,
    "filters": [
      {
        "capacity_limit": 363408,
        "closed_at": null,
        "created_at": 1776432374,
        "download_url": "/filters/filter-1776432374706800409",
        "encoding": "base64",
        "file_name": "filter-1776432374706800409.bloom",
        "filter_id": "filter-1776432374706800409",
        "inserted_count": 101612,
        "k": 32,
        "m_bits": 16777216,
        "sha256_base64": "YlRf8t2Wsb+Ybl+jS0jjj3nYUgjHW+cDX7i/jBDpqrA=",
        "status": "active",
        "updated_at": 1776432663,
        "window_start_max": 2641210259,
        "window_start_min": 1776432446
      }
    ],
    "public_base_url": null,
    "service": "bfilter",
    "updated_at": 1776686451,
    "version": 1
  },
  "ok": true
}
```

Este é o bloom filter principal. Para executar o nosso teste você deve rodar um outro bloom filter secundário que será usado para testar os eventos de falso positivo. Na prática, em um sistema operacional, este segundo serviço de bloom filter não é necessário.

Abra um outro terminal e digite:

```
BFILTER_BIND_ADDR='0.0.0.0:8081' BFILTER_ENABLE_TEST_API=1  
BFILTER_ADMIN_TOKEN="dev-admin-token" cargo run
```

Você verá uma tela muito semelhante a que mostramos anteriormente, exceto pelo fato do serviço estar rodando na porta 8081 ao invés da porta 8080. No navegador você verá algo parecido com isto:



```
{  
  "manifest": {  
    "active_filter_id": "filter-1776704129424975823",  
    "false_positive_power": 32,  
    "filters": [  
      {  
        "capacity_limit": 709,  
        "closed_at": 1776704127,  
        "created_at": 1776704126,  
        "download_url": "/filters/filter-1776704126627113201",  
        "encoding": "base64",  
        "file_name": "filter-1776704126627113201.bloom",  
        "filter_id": "filter-1776704126627113201",  
        "inserted_count": 750,  
        "k": 32,  
        "m_bits": 32768,  
        "sha256_base64": "yw2XR0ro/krVbmz4GQidDpeGc3IV95ryd7qgFj3iMIE=",  
        "status": "closed",  
        "updated_at": 1776704127,  
        "window_start_max": 1809017727,  
        "window_start_min": 1776704126  
      },  
      {  
        "capacity_limit": 709,  
        "closed_at": 1776704128,  
        "created_at": 1776704127,  
        "download_url": "/filters/filter-1776704127482197399",  
        "encoding": "base64",  
        "file_name": "filter-1776704127482197399.bloom",  
        "filter_id": "filter-1776704127482197399",  
        "inserted_count": 750,  
        "k": 32,  
        "m_bits": 32768,  
        "sha256_base64": "8SBZ1YvG+8SFpGdtXiDclPbwYaApa374tuNqGnP+ppg=",  
        "status": "closed",  
        "updated_at": 1776704128,  
        "window_start_max": 1809017727,  
        "window_start_min": 1776704127  
      }  
    ]  
  }  
}
```

Esta segunda instância do bloom filter é usada apenas neste teste. Num sistema de produção o Emissor pode escolher se vai disponibilizar a API do bloom filter ou somente um arquivo estático (o que garante uma melhor não-rastreabilidade já que o Verificador pode consultar o bloom filter off-line através de uma API semelhante). Para fins de testes o serviço de API é mais cômodo.

2. Executar o teste Benchmark

Para executar este teste basta clonar e abrir a pasta “ssi-electron-revoke” e executar o seguinte comando:

```
npm run bench:all
```

O teste demora quase 5 minutos para rodar porque escreve no ledger criando schemas, CredDefs, DIDs, escreve vários ATTRIBs para registrar o vetor K e a âncora do arquivo Manifesto.

Nosso notebook de teste é um Lenovo com processador core 12th Gen Intel Core i5-12450H de 12 núcleos e 16GB de memória RAM, com o sistema operacional Linux Mint 22.2 Zara.

Para ver dados do seu computador use o comando abaixo:

```
inxi -F
```

No final o teste gera um arquivo chamado **paper-summary.md** que contém todos os dados do teste. Ao final do teste você verá algo como:

Tabela 1. Emissão da Credencial Revogável

Janelas	Experimentos	Janelas válidas	Confirmação	Emissão (ms)	Envelope Holder (bytes)	Bundle (bytes)	Manifesto (bytes)	Chunks K
100	3	100	10	76.663	78839	33617	614	11
365	3	365	10	202.965	198715	111060	614	11
1000	3	1000	10	436.316	486043	296694	614	11
5000	3	5000	10	1991.9	2295742	1465958	614	11
10000	3	10000	10	3938.547	4557387	2927182	614	11

Tabela 2. Latência de Verificação

Janelas	Cenário	Experimentos	Warmups	Janela revogação	Mediana (ms)	P95 (ms)	Janelas consultadas	Modo dominante
100	not_revoked	10	1	-	108.694	114.579	18	binary_window_search
100	revoked_early	10	1	0	67.188	68.954	12	binary_window_search
100	revoked_middle	10	1	50	51.676	59.855	11	binary_window_search
100	revoked_late	10	1	99	20.56	24.768	8	binary_window_search
365	not_revoked	10	1	-	130.842	139.561	20	binary_window_search
365	revoked_early	10	1	0	100.737	109.237	16	binary_window_search
365	revoked_middle	10	1	182	33.105	40.124	10	binary_window_search
365	revoked_late	10	1	364	35.731	40.902	10	binary_window_search
1000	not_revoked	10	1	-	172.761	179.972	21	binary_window_search
1000	revoked_early	10	1	0	149.5	157.655	18	binary_window_search
1000	revoked_middle	10	1	500	136.228	146.546	17	binary_window_search
1000	revoked_late	10	1	999	57.117	63.873	11	binary_window_search
5000	not_revoked	10	1	-	446.219	473.231	24	binary_window_search
5000	revoked_early	10	1	0	453.826	475.855	24	binary_window_search
5000	revoked_middle	10	1	2500	438.164	457.121	23	binary_window_search
5000	revoked_late	10	1	4999	232.335	243.343	14	binary_window_search
10000	not_revoked	10	1	-	786.833	815.671	25	binary_window_search
10000	revoked_early	10	1	0	836.054	869.961	26	binary_window_search
10000	revoked_middle	10	1	5000	782.929	812.413	25	binary_window_search
10000	revoked_late	10	1	9999	437.75	452.734	15	binary_window_search

O arquivo final tem 2 páginas e 7 tabelas de dados que foram usadas para extrair os dados que estão na tabela do nosso paper “BlindRevoke: Um Mecanismo de Revogação para SSI”.